

银 轮 信 息 开 发 规 约

版 本：1.5

编 制 人：许晓南

编制日期：2021 年 9 月 8 日

更新日期：2025 年 7 月 3 日

参考资料：《阿里巴巴 Java 开发手册（嵩山版）》

版本号	制定人	更新日期	备注
0.5	许晓南、余其壮、余宽	2021.9.8	选择常用的基础规约作为第一版规范
1.0	许晓南	2021.9.13	补充更多常规规约
1.1	许晓南	2021.9.16	部门学习后，对部分规约进行调整
1.2	余其壮、许晓南	2021.12.23	增加 MySQL 优化搜索相关内容若干条。 增加开发设计规约
1.3	许晓南	2022.1.24	增加定时任务业务规约
1.4	许晓南	2024.1.20	增加业务规约，所有列表需要支持列排序
1.5	许晓南	2025.7.3	增加安全规约，增加第三方对接规约，增加 git 日志提交规范

目录

前言	4
一、 编程规约.....	4
(一) 注释规约.....	4
二、编程规约.....	6
(一) 命名风格.....	6
(二) 常量定义.....	9
(三) OOP 规约.....	10
(四) 日期时间.....	13
(五) 集合处理.....	14
(六) 并发处理.....	15
(七) 控制语句.....	16
(八) 前后端规约.....	19
三、业务规约.....	20
(一) 表字段.....	20
四、MySQL 数据库.....	21
(一) 建表规约.....	21
(二) SQL 语句.....	22
(三) ORM 映射.....	23

前言

随着公司的发展，开发部人员不断增加，每个开发人员工作经历不同，在代码的编写习惯上存在较大的差异，出现问题时不易进行快速排查，若开发人员变动也不利进行工作交接，包括新项目立项时，不同的开发习惯或业务习惯为团队的磨合也增加了非必要的成本。

本着提高协作效率、规范公司项目代码及方便项目后续的维护的想法，部门组织学习了《阿里巴巴 Java 开发手册（嵩山版）》（2021 年版本），对其中符合公司当前开发环境的内容进行了摘选，并结合公司实际情况进行了内容的补充与删减，保留内容均为日常使用频率较高、且必须遵守的基本编码规范。希望在本规范推出以后，开发人员以此规范为标准进行代码的编写，并在团队协作过程中提出更多有利于部门发展的内容，作为追加规约进行补充。

补充：阿里巴巴有推出代码规范扫描插件（Alibaba Java Coding Guidelines），开发人员可在 IDE 上进行引入并进行代码的扫描，对不符合规约的代码进行处理，建议每个后端开发人员均进行安装。

一、安全规约

（一）信息安全

1、【强制】第三方接口信息安全

系统存在第三方系统对接时，对第三方系统调用信息应予以保密，避免泄露

说明：

- 1、使用前台界面配置信息时，包含密码或者 key 值的内容应该默认使用星号*隐藏，并支持取消隐藏查看
- 2、如果接口没有权限校验的情况下，应不予返回密码或者 key 值，前台通过标识，使用户明确是否已添加配置
- 3、后台存储使用加密方式存贮，使用时再进行解密，可使用 Jasypt，启动时指定盐值

2、【强制】接口文档安全

系统中使用了 swagger 或者类型接口文档插件时，必须添加访问权限，强密码，或者在正式环境发布时关闭该文档的访问

说明：

- 1、推荐使用加密访问的形式，可以增加安全性，也保留调试便利性
- 2、目前大部分系统应该没有接口级的权限校验，暴露接口文档后，在用户拿到 token 后即可越权调用

(二) 部署安全

1、【强制】数据库部署

数据库作为系统的核心，应该保持高保密性，防止数据泄露，尤其是包含了价格或其他敏感信息内容时

说明：

- 1、密码均使用强密码
- 2、MySQL 端口应修改成非默认端口，增加安全性。Oracle 及 SqlServer 视项目要求判断是否做端口变更
- 3、数据库必须每日进行至少一次的备份，推荐使用 navicat12 以上版本进行定期备份，要求客户做异地备份
- 4、系统使用账号需要单独建立，仅一台服务器使用数据库时应限定仅本机或者指定 IP 访问。
- 5、只读访问的账号应在账号层面配置只读权限，特别提供第三方读取时必须提供只读账号

2、【强制】运行可靠性

系统在运行时，应保持高效稳定，服务器存在异常关闭时，应能在服务器启动后恢复业务使用

说明：

- 1、系统用到的所有软件应该配置成开机自启形式
- 2、Apache、nginx、redis、tomcat 等均可配置自动或借助 nssm 安装成服务自启
- 3、对于用到的第三方软件，也尽量通过 window 任务计划程序配置启动自启；确实无法实现自启的程序，应告知用户进行主动启动

二、编程规约

(一) 注释规约

1、**【强制】** 类、类属性、类方法的注释必须使用 Javadoc 规范，使用/**内容*/格式，不得使用// xxx 方式

说明：在 IDE 编辑窗口中，Javadoc 方式会提示相关注释，生成 Javadoc 可以正确输出相应注释；在 IDE 中，工程调用方法时，不进入方法即可悬浮提示方法、参数、返回值的意义，提高阅读效率。

2、**【强制】** 所有的抽象方法（包括接口中的方法）必须要用 Javadoc 注释、除了返回值、参数、异常说明外，还必须指出该方法做什么事情，实现什么功能。

说明：对子类的实现要求，或者调用注意事项，请一并说明。

3、**【强制】** 所有的类都必须添加创建者和创建日期。

说明：在设置模板时，注意 IDEA 的@author 为`\${USER}`，而 eclipse 的@author 为`\${user}`，大小写有。作者需要以中文名进行注释

正例：

```
/**
 * @Author: 许晓南.
 * @Date: 2021/9/7
 * @Version: 1.0
 */
```

4、**【强制】** 方法内部单行注释，在被注释语句上方另起一行，使用//注释。方法内部多行注释使用/* */注释，注意与代码对齐。

5、**【强制】** 所有的枚举类型字段必须要有注释，说明每个数据项的用途。

6、**【强制】** 代码修改的同时，注释也要进行相应的修改，尤其是参数、返回值、异常、核心逻辑等的修改。

7、**【推荐】** 在类中删除未使用的任何字段、方法、内部类；在方法中删除未使用的任何参数声明与内部变量。

8、**【参考】** 谨慎注释掉代码。在上方详细说明，而不是简单地注释掉。如果无用，

则删除。

说明：代码被注释掉有两种可能性：

- 1) 后续会恢复此段代码逻辑。
- 2) 永久不用。前者如果没有备注信息，难以知晓注释动机。后者建议直接删掉即可，假如需要查阅历史代码，登录代码仓库即可。

9、【参考】对于注释的要求：

第一：能够准确反映设计思想和代码逻辑；

第二能够描述业务含义，使别的程序员能够迅速了解到代码背后的信息。完全没有注释的大段代码对于阅读者形同天书，注释是给自己看的，即使隔很长时间，也能清晰理解当时的思路；注释也是给继任者看的，使其能够快速接替自己的工作。

10、【参考】特殊注释标记，请注明标记人与标记时间。注意及时处理这些标记，通过标记扫描，经常清理此类标记。线上故障有时候就是来源于这些标记处的代码。

1) 待办事宜 (TODO)：(标记人, 标记时间, [预计处理时间]) 表示需要实现，但目前还未实现的功能。这实际上是一个 Javadoc 的标签，目前的 Javadoc 还没有实现，但已经被广泛使用。只能应用于类，接口和方法（因为它是一个 Javadoc 标签）。

2) 错误，不能工作 (FIXME)：(标记人, 标记时间, [预计处理时间]) 在注释中用 FIXME 标记某代码是错误的，而且不能工作，需要及时纠正的情况。

三、编程规约

(一) 命名风格

1、**【强制】** 代码中的命名均不能以下划线或美元符号开始，也不能以下划线或美元符号结束。

反例： `_name` / `name` / `$name` / `name_` / `name$` / `name__`

2、**【强制】** 所有编程相关的命名严禁使用拼音与英文混合的方式，更不允许直接使用中文的方式。

说明：正确的英文拼写和语法可以让阅读者易于理解，避免歧义。注意，纯拼音命名方式更要避免采用。

正例： `ali` / `alibaba` / `taobao` / `cainiao` / `aliyun` / `youku` / `hangzhou` 等国际通用的名称，可视同英文。

反例： `DaZhePromotion` [打折] / `getPingfenByName()` [评分] / `int 某变量 = 3`

3、**【强制】** 类名使用 UpperCamelCase 风格，但以下情形例外：`DO` / `BO` / `DTO` / `VO` / `AO` / `PO` / `UID` 等。

正例： `ForceCode` / `UserDO` / `HtmlDTO` / `XmlService` / `TcpUdpDeal` / `TaPromotion`

反例： `forcecode` / `UserDo` / `HTMLDto` / `XMLService` / `TCPUDPDeal` / `TAPromotion`

4、**【强制】** 方法名、参数名、成员变量、局部变量都统一使用 lowerCamelCase 风格。

正例： `localValue` / `getHttpMessage()` / `inputUserId`

5、**【强制】** 常量命名全部大写，单词间用下划线隔开，力求语义表达完整清楚，不要嫌名字长。

正例： `MAX_STOCK_COUNT` / `CACHE_EXPIRED_TIME`

反例： `MAX_COUNT` / `EXPIRED_TIME`

6、**【强制】** 抽象类命名使用 `Abstract` 或 `Base` 开头；异常类命名使用 `Exception` 结尾；测试类命名以它要测试的类的名称开始，以 `Test` 结尾。

7、【强制】类型与中括号紧挨相连来表示数组。

正例：定义整形数组 `int[] arrayDemo;`

反例：在 `main` 参数中，使用 `Stringargs[]`来定义。

8、【强制】包名统一使用小写，点分隔符之间有且仅有一个自然语义的英语单词。包名统一使用单数形式，但是类名如果有复数含义，类名可以使用复数形式。**正例：**应用工具类包名为 `com.alibaba.ei.kunlun.aap.util`、类名为 `MessageUtils`（此规则参考 `spring` 的框架结构）。

9、【强制】避免在子父类的成员变量之间、或者不同代码块的局部变量之间采用完全相同的命名，使可读性降低。

说明：子类、父类成员变量名相同，即使是 `public` 类型的变量也是能够通过编译，而局部变量在同一方法内的不同代码块中同名也是合法的，但是要避免使用。对于非 `setter/getter` 的参数名称也要避免与成员变量名称相同。

反例：

```
public class ConfusingName {
    public int stock;

    // 非 setter/getter 的参数名称，不允许与本类成员变量同名

    public void get(String alibaba) {
        if (condition) {
            final int money = 666;
        }
        for (int i = 0; i < 10; i++) {
            // 在同一方法体中，不允许与其它代码块中的 money 命名相同

            final int money = 15978;
        }
    }
}

class Son extends ConfusingName {
    // 不允许与父类的成员变量名称相同

    public int stock;
}
```

10、【强制】杜绝完全不规范的缩写，避免望文不知义。**反例：**"AbstractClass"缩写命名成 `AbsClass`；"condition"缩写命名成 `condi`，此类随意缩写严重降低了代码的可阅读性。

11、**【推荐】** 在常量与变量的命名时，表示类型的名词放在词尾，以提升辨识度。

正例： startTime / workQueue / nameList / TERMINATED_THREAD_COUNT

反例： startedAt / QueueOfWork / listName / COUNT_TERMINATED_THREAD

12、**【推荐】** 如果模块、接口、类、方法使用了设计模式，在命名时需体现出具体模式。

说明： 将设计模式体现在名字中，有利于读者快速理解架构设计理念。

正例：

```
public class OrderFactory;  
public class LoginProxy;  
public class ResourceObserver;
```

13、**【推荐】** 接口类中的方法和属性不要加任何修饰符号（public 也不要加），保持代码的简洁性，并加上有效的 Javadoc 注释。尽量不要在接口里定义变量，如果一定要定义变量，确定与接口方法相关，并且是整个应用的基础常量。

正例： 接口方法签名 void commit();
接口基础常量 String COMPANY = "alibaba";

反例： 接口方法定义 public abstract void f();

说明： JDK8 中接口允许有默认实现，那么这个 default 方法，是对所有实现类都有价值的默认实现。

接口和实现类的命名有两套规则：

1) **【强制】** 对于 Service 和 DAO 类，基于 SOA 的理念，暴露出来的服务一定是接口，内部的实现类用 Impl 的后缀与接口区别。

正例： CacheServiceImpl 实现 CacheService 接口。

2) **【推荐】** 如果是形容能力的接口名称，取对应的形容词为接口名（通常是-able 的形容词）。

正例： AbstractTranslator 实现 Translatable 接口。

14、**【参考】** 枚举类名带上 Enum 后缀，枚举成员名称需要全大写，单词间用下划线隔开。

说明： 枚举其实就是特殊的常量类，且构造方法被默认强制是私有。

正例： 枚举名字为 ProcessStatusEnum 的成员名称：SUCCESS / UNKNOWN_REASON。

15、各层命名规约：

A) **【强制】** Service/DAO 层方法命名规约（按钮权限命名同接口名称）

- 1) 获取单个对象的方法用 get 做前缀。
- 2) 获取多个对象的方法用 list 做前缀，复数结尾，如：listObjects。
- 3) 获取统计值的方法用 count 做前缀。
- 4) 插入的方法用 insert 做前缀。
- 5) 删除的方法用 delete 做前缀。
- 6) 修改的方法用 update 做前缀。

7) 启用停用的方法用 `updateStatus` 做前缀；如启用停用其他业务状态，如流程状态，使用 `updateProcessStatus` 做前缀。

8) 导入方法使用 `import` 做前缀。

9) 导出使用 `export` 做前缀。

10) 下载模板使用 `download` 做前缀。

B) 【参考】领域模型命名规约

1) 数据对象：`xxxDO`，`xxx` 即为数据表名。

2) 数据传输对象：`xxxDTO`，`xxx` 为业务领域相关的名称。

3) 展示对象：`xxxVO`，`xxx` 一般为网页名称。

4) POJO 是 DO/DTO/BO/VO 的统称，禁止命名成 `xxxPOJO`。

(二) 常量定义

1、【强制】在 `long` 或者 `Long` 赋值时，数值后使用大写的 `L`，不能是小写的 `l`，小写容易跟数字混淆，造成误解。

说明：`Long a = 2l;` 写的是数字的 `21`，还是 `Long` 型的 `2`。

2、【推荐】不允许任何魔法值（即未经预先定义的常量）直接出现在代码中。

反例：

//本例中同学 A 定义了缓存的 `key`，然后缓存提取的同学 B 使用了 `Id#taobao` 来提取，少了下划线，导致故障。

```
String key = "Id#taobao_" + tradelId;  
cache.put(key, value);
```

3、【推荐】不要使用一个常量类维护所有常量，要按常量功能进行归类，分开维护。

说明：大而全的常量类，杂乱无章，使用查找功能才能定位到修改的常量，不利于理解，也不利于维护。

正例：

缓存相关常量放在类 `CacheConsts` 下；系统配置相关常量放在类 `ConfigConsts` 下。

4、【推荐】如果变量值仅在一个固定范围内变化用 `enum` 类型来定义。

说明：如果存在名称之外的延伸属性应使用 `enum` 类型，下面正例中的数字就是延伸信息，表示一年中的第几个季节。

正例：

```
public enum SeasonEnum {  
    SPRING(1), SUMMER(2), AUTUMN(3), WINTER(4);  
    private int seq;  
    SeasonEnum(int seq) {  
        this.seq = seq;  
    }  
}
```

```

    public int getSeq () {
        return seq;
    }
}

```

(三) OOP 规约

(Object Oriented Programming,指面向对象程序设计)

1、**【强制】** 避免通过一个类的对象引用访问此类的静态变量或静态方法，无谓增加编译器解析成本，直接用类名来访问即可。

```

public class ResultConstant {

    // 响应业务状态

    private Integer status = 200;
}

```

正例：使用静态变量 ResultConstant.status.equals(result)

反例：使用静态变量 ResultConstant rc = new ResultConstant()
 rc.status.equals(result)

2、**【强制】** 所有的覆写方法，必须加@Override 注解。

说明：getObject()与 get0bject()的问题。一个是字母的 O，一个是数字的 0，加@Override 可以准确判断是否覆盖成功。另外，如果在抽象类中对方法签名进行修改，其实现类会马上编译报错。

3、**【强制】** 相同参数类型，相同业务含义，才可以使用 Java 的可变参数，避免使用 Object。

说明：可变参数必须放置在参数列表的最后。（提倡同学们尽量不用可变参数编程）

正例：public List<User>listUsers(String type, Long...ids){...}

4、**【强制】** 外部正在调用或者二方库依赖的接口，不允许修改方法签名，避免对接口调用方产生影响。接口过时必须加@Deprecated 注解，并清晰地说明采用的新接口或者新服务是什么。

5、**【强制】** 不能使用过时的类或方法。

说明：java.net.URLDecoder 中的方法 decode(String encodeStr) 这个方法已经过时，应该使用双参数 decode(String source, String encode)。接口提供方既然明确是过时接口，那么有义务同时提供新的接口；作为调用方来说，有义务去考证过时方法的新实现是什么。

6、**【强制】** Object 的 equals 方法容易抛空指针异常，应使用常量或确定有值的对象来调用 equals。

正例： "test".equals(object);

反例： object.equals("test");

说明： 推荐使用 java.util.Objects#equals (JDK7 引入的工具类) 。

7、**【强制】** 所有整型包装类对象之间值的比较，全部使用 equals 方法比较。

说明： 对于整型都使用 Integer 来接收，比较时使用 equals 进行比较。

8、**【强制】** 浮点数均使用 BigDecimal 来接收，所有计算均使用 BigDecimal 自带的方法进行，数据库小数均使用 decimal。

正例：

使用 BigDecimal 来定义值，再进行浮点数的运算操作。

```
public static void main(String[] args)
{
    BigDecimal a = new
    BigDecimal("1.0");
    BigDecimal b = new
    BigDecimal("0.9");
    BigDecimal c = new BigDecimal("0.8");
    BigDecimal x = a.subtract(b);
    BigDecimal y = b.subtract(c);
    if (x.equals(y)) {
        System.out.println("true");
    }
}
```

9、关于基本数据类型与包装数据类型的使用标准如下：

1) **【强制】** 所有的 POJO 类属性必须使用包装数据类型。

正例：

```
public class Person
{
    private String
    name;
    private
    Integer age;
}
```

反例：

```
public class Person
{
    private String
    name;
    private int
    age;
}
```

2) **【推荐】** 所有的局部变量使用基本数据类型。

10、**【强制】** 定义 DO/DTO/VO 等 POJO 类时，不要设定任何属性默认值。

反例： POJO 类的 createTime 默认值为 new Date()，但是这个属性在数据提取时并没有置入具体值，在更新其它字段时又附带更新了此字段，导致创建时间被修改成当前时间。

11、**【强制】** 序列化类新增属性时，请不要修改 `serialVersionUID` 字段，避免反序列化失败；如果完全不兼容升级，避免反序列化混乱，那么请修改 `serialVersionUID` 值。

说明：注意 `serialVersionUID` 不一致会抛出序列化运行时异常。

12、**【强制】** 构造方法里面禁止加入任何业务逻辑，如果有初始化逻辑，请放在 `init` 方法中。

13、**【推荐】** 使用索引访问用 `String` 的 `split` 方法得到的数组时，需做最后一个分隔符后有无内容的检查，否则会有抛 `IndexOutOfBoundsException` 的风险。

14、**【推荐】** 当一个类有多个构造方法，或者多个同名方法，这些方法应该按顺序放置在一起，便于阅读，此条规则优先于下一条。

15、**【推荐】** 类内方法定义的顺序依次是：公有方法或保护方法 > 私有方法 > `getter / setter` 方法。

说明：公有方法是类的调用者和维护者最关心的方法，首屏展示最好；保护方法虽然只是子类关心，也可能是“模板设计模式”下的核心方法；而私有方法外部一般不需要特别关心，是一个黑盒实现；因为承载的信息价值较低，所有 `Service` 和 `DAO` 的 `getter/setter` 方法放在类体最后。

16、**【强制】** 循环体内，字符串的连接方式，使用 `StringBuilder` 的 `append` 方法进行扩展。

说明：下例中，反编译出的字节码文件显示每次循环都会 `new` 出一个 `StringBuilder` 对象，然后进行 `append` 操作，最后通过 `toString` 方法返回 `String` 对象，造成内存资源浪费。

反例：

```
public static void main(String[] args) {
    String str = "start";
    for (int i = 0; i < 100; i++) {
        str = str + "hello";
    }
}
```

17、**【推荐】** `final` 可以声明类、成员变量、方法、以及本地变量，下列情况使用 `final` 关键字：

- 1) 不允许被继承的类，如：`String` 类。
- 2) 不允许修改引用的域对象，如：`POJO` 类的域变量。
- 3) 不允许被覆写的方法，如：`POJO` 类的 `setter` 方法
- 4) 不允许运行过程中重新赋值的局部变量。

-
- 5) 避免上下文重复使用一个变量，使用 `final` 可以强制重新定义一个变量，方便更好地进行重构。

18、【推荐】类成员与方法访问控制从严：

- 1) 如果不允许外部直接通过 `new` 来创建对象，那么构造方法必须是 `private`。
- 2) 工具类不允许有 `public` 或 `default` 构造方法。
- 3) 类非 `static` 成员变量并且与子类共享，必须是 `protected`。
- 4) 类非 `static` 成员变量并且仅在本类使用，必须是 `private`。
- 5) 类 `static` 成员变量如果仅在本类使用，必须是 `private`。
- 6) 若是 `static` 成员变量，考虑是否为 `final`。
- 7) 类成员方法只供类内部调用，必须是 `private`。
- 8) 类成员方法只对继承类公开，那么限制为 `protected`。

说明：任何类、方法、参数、变量，严控访问范围。过于宽泛的访问范围，不利于模块解耦。思考：如果是一个 `private` 的方法，想删除就删除，可是一个 `public` 的 `service` 成员方法或成员变量，删除一下，不得手心冒点汗吗？变量像自己的小孩，尽量在自己的视线内，变量作用域太大，无限制的到处跑，那么你会担心的。

(四) 日期时间

1、【强制】日期格式化时，传入 `pattern` 中表示年份统一使用小写的 `y`。

说明：日期格式化时，`yyyy` 表示当天所在的年，而大写的 `YYYY` 代表是 `week in which year` (JDK7 之后引入的概念)，意思是当天所在的周属于的年份，一周从周日开始，周六结束，只要本周跨年，返回的 `YYYY` 就是下一年。

正例：表示日期和时间的格式如下所示：

```
new SimpleDateFormat("yyyy-MM-dd HH:mm:ss")
```

2、【强制】在日期格式中分清楚大写的 `M` 和小写的 `m`，大写的 `H` 和小写的 `h` 分别指代的意义。说明：日期格式中的这两对

字母表意如下：

- 1) 表示月份是大写的 `M`；
- 2) 表示分钟则是小写的 `m`；
- 3) 24 小时制的是大写的 `H`；
- 4) 12 小时制的则是小写的 `h`。

3、【强制】获取当前毫秒数：`System.currentTimeMillis()`；而不是 `new Date().getTime()`。

说明：如果想获取更加精确的纳秒级时间值，使用 `System.nanoTime` 的方式。在 JDK8 中，针对统计时间等场景，推荐使用 `Instant` 类。

4、【强制】不允许在程序任何地方中使用：

- 1) java.sql.Date
- 2) java.sql.Time
- 3) java.sql.Timestamp。

说明：第 1 个不记录时间，getHours()抛出异常；第 2 个不记录日期，getYear()抛出异常；第 3 个在构造方法 super((time/1000)*1000)，fastTime 和 nanos 分开存储秒和纳秒信息。

反例： java.util.Date.after(Date)进行时间比较时，当入参是 java.sql.Timestamp 时，会触发 JDKBUG(JDK9 已修复)，可能导致比较时的意外结果。

5、【强制】不要在程序中写死一年为 365 天，避免在公历闰年时出现日期转换 错误或程序逻辑错误。

正例：

```
// 获取今年的天数
int daysOfThisYear = LocalDate.now().lengthOfYear();
// 获取指定某年的天数
LocalDate.of(2011, 1, 1).lengthOfYear();
```

反例：

```
// 第一种情况：在闰年 366 天时，出现数组越界异常
int[] dayArray = new int[365];
// 第二种情况：一年有效期的会员制，今年 1 月 26 日注册，硬编码 365
返回的却是 1 月 25 日
Calendar calendar = Calendar.getInstance();
calendar.set(2020, 1, 26);
calendar.add(Calendar.DATE, 365);
```

6、【推荐】避免公历闰年 2 月问题。闰年的 2 月份有 29 天，一年后的那一天不可能是 2 月 29 日。

7、【推荐】使用枚举值来指代月份。如果使用数字，注意 Date，Calendar 等日期相关类的月份 month 取值在 0-11 之间。

说明：参考 JDK 原生注释，Month value is 0-based. e.g., 0 for January.

正例： Calendar.JANUARY, Calendar.FEBRUARY, Calendar.MARCH 等来指代相应月份来进行传参或比较。

(五) 集合处理

1、【强制】判断所有集合内部的元素是否为空，使用 isEmpty()方法，而不是

size()==0 的方式。

- 2、**【强制】不要在** foreach 循环里进行元素的 remove/add 操作。remove 元素请使用 Iterator 方式，如果并发操作，需要对 Iterator 对象加锁。

正例：

```
List<String> list = new ArrayList<>();
list.add("1");
list.add("2");
Iterator<String> iterator = list.iterator();
while (iterator.hasNext()) {
    String item = iterator.next();

    if (删除元素的条件) {
        iterator.remove();
    }
}
```

反例：

```
for (String item : list) {
    if ("1".equals(item)) {
        list.remove(item);
    }
}
```

说明：以上代码的执行结果肯定会出乎大家的意料，那么试一下把“1”换成“2”，会是同样的结果吗？for 循环中移除了元素导致下标错误，代码执行会报错。

(六) 并发处理

- 1、**【强制】SimpleDateFormat** 是线程不安全的类，一般不要定义为 static 变量，如果定义为 static，必须加锁，或者使用 DateUtils 工具类。**正例：**注意线程安全，使用 DateUtils。亦推荐如下处理：

```
private static final ThreadLocal<DateFormat> df = new
    ThreadLocal<DateFormat>() {
        @Override
        protected DateFormat initialValue() {
            return new SimpleDateFormat("yyyy-MM-dd");
        }
    }
```

说明：如果是 JDK8 的应用，可以使用 Instant 代替 Date，LocalDateTime 代替 Calendar，DateTimeFormatter 代替 SimpleDateFormat，官方给出的解释：simple beautiful strong immutablethread-safe。

(七) 控制语句

1、**【强制】** 在一个 switch 块内，每个 case 要么通过 continue/break/return 等来终止，要么注释说明程序将继续执行到哪一个 case 为止；在一个 switch 块内，都必须包含一个 default 语句并且放在最后，即使它什么代码也没有。

说明：注意 break 是退出 switch 语句块，而 return 是退出方法体。

2、**【强制】** 当 switch 括号内的变量类型为 String 并且此变量为外部参数时，必须先进行 null 判断。

反例：如下的代码输出是什么？

```
public class SwitchString {
    public static void main(String[] args)
    { method(null);
    }
    public static void method(String param) {
        switch (param) {
            // 肯定不是进入这里
            case "sth":
                System.out.println("it's sth");
                break;
            // 也不是进入这里
            case "null":
                System.out.println("it's null");
                break;
            // 也不是进入这里
            default:
                System.out.println("default");
        }
    }
}
```

3、**【强制】** 在 if/else/for/while/do 语句中必须使用大括号

说明：即使只有一行代码，也禁止不采用大括号的编码方式：if (condition) statements;

4、**【强制】** 三目运算符 condition? 表达式 1: 表达式 2 中，高度注意表达式 1 和 2 在类型对齐时，可能抛出因自动拆箱导致的 NPE 异常。说明：以下两种场景会触发类型对齐的拆箱操作：

1) 表达式 1 或表达式 2 的值只要有一个是原始类型。

2) 表达式 1 或表达式 2 的值的类型不一致，会强制拆箱升级成表示范围更大的那个类型。

反例：

```
Integer a = 1;
Integer b = 2;
Integer c = null;
Boolean flag = false;

// a*b 的结果是 int 类型，那么 c 会强制拆箱成 int 类型，抛出 NPE 异常
Integer result = (flag ? a * b : c);
```

5、【强制】在高频并发场景中，避免使用“等于”判断作为中断或退出的条件。

说明：如果并发控制没有处理好，容易产生等值判断被“击穿”的情况，使用大于或小于的区间判断条件来代替。

反例：

判断剩余奖品数量等于 0 时，终止发放奖品，但因为并发处理错误导致奖品数量瞬间变成了负数，这样的话，活动无法终止。

6、【推荐】表达异常的分支时，少用 if-else 方式，这种方式可以改写成：

```
if (condition) {
    ...
    return obj;
}

// 接着写 else 的业务逻辑代码；
```

说明：如果非使用 if()...else if()...else...方式表达逻辑，避免后续代码维护困难，请勿超过 3 层。

正例：超过 3 层的 if-else 的逻辑判断代码可以使用卫语句、策略模式、状态模式等来实现，其中卫语句示例如下：

```
public void findBoyfriend (Man man) {
    if (man.isUgly()) {
        System.out.println("本姑娘是外貌协会的资深会员");
        return;
    }
    if (man.isPoor()) {
        System.out.println("贫贱夫妻百事哀");
        return;
    }
}
```

```

    }
    if (man.isBadTemper()) {
        System.out.println("银河有多远，你就给我滚多远");
        return;
    }
    System.out.println("可以先交往一段时间看看");
}

```

7、【推荐】除常用方法（如 getXxx/isXxx）等外，不要在条件判断中执行其它复杂的语句，将复杂逻辑判断的结果赋值给一个有意义的布尔变量名，以提高可读性。

说明：很多 if 语句内的逻辑表达式相当复杂，与、或、取反混合运算，甚至各种方法纵深调用，理解成本非常高。如果赋值一个非常好理解的布尔变量名字，则是件令人赏心悦目的事情。

正例：

```

// 伪代码如下
final boolean existed = (file.open(fileName, "w") !=
null) && (...) || (...);
if (existed) {
    ...
}

```

反例：

```

public final void acquire ( long arg){
    if (!tryAcquire(arg) &&
        acquireQueued(addWaiter(Node.EXCLUSIVE), arg))
    {
        selfInterrupt();
    }
}

```

8、【推荐】不要在其它表达式（尤其是条件表达式）中，插入赋值语句。

说明：赋值点类似于人体的穴位，对于代码的理解至关重要，所以赋值语句需要清晰地单独成为一行。

反例：

```
public Lock getLock(boolean fair) {  
    // 算术表达式中出现赋值操作，容易忽略 count 值已经被改变  
    threshold = (count = Integer.MAX_VALUE) - 1;  
    // 条件表达式中出现赋值操作，容易误认为是 sync==fair  
    return (sync = fair) ? new FairSync() : new  
    NonfairSync();  
}
```

9、【推荐】避免采用取反逻辑运算符。

10、【推荐】公开接口需要进行入参保护，尤其是批量操作的接口。

反例：某业务系统，提供一个用户批量查询的接口，API 文档上有说最多查多少个，但接口实现上没做任何保护，导致调用方传了一个 1000 的用户 id 数组过来后，查询信息后，内存爆满了。

(八) 前后端规约

1、【强制】在前后端交互的 JSON 格式数据中，所有的 key 必须为小写字母开始的 lowerCamelCase 风格，符合英文表达习惯，且表意完整。

正例：errorCode / errorMessage / assetStatus / menuList / orderList / configFlag 反例：
ERRORCODE / ERROR_CODE / error_message / error-message / errormessage /
ErrorMessage / msg

2、【强制】对于需要使用超大整数的场景，服务端一律使用 String 字符串类型返回，禁止使用 Long 类型；如果是需要使用小数的场景，服务端一律使用 String 字符串类型来返回，防止精度丢失。

3、【强制】HTTP 请求通过 URL 传递参数时，不能超过 2048 字节。

说明：不同浏览器对于 URL 的最大长度限制略有不同，并且对超出最大长度的处理逻辑也有差异，2048 字节是取所有浏览器的最小值。

反例：某业务将退货的商品 id 列表放在 URL 中作为参数传递，当一次退货商品数量过多时，URL 参数超长，传递到后端的参数被截断，导致部分商品未能正确退货。

4、【推荐】前后端的时间格式统一为"yyyy-MM-dd HH:mm:ss"，统一为 GMT。

(九) 前端规约

1、【强制】所有弹窗中的数据，在弹窗关闭时必须清除里面的数据。

反例：

在编辑 A 数据后回到列表，点开了 B 数据的编辑，由于网速过慢，在通过 ID 请求 B 数据详情时，A 数据的详情还保存在弹窗中，用户直接修改并进行了保存，导致 A 的详细数据覆盖到了 B 数据上。

2、【强制】页面初始化时或打开页面获取必要数据时，需要请求后台获取一些初始化数据，必须添加蒙板（遮罩），控制用户不能进行业务操作，必须等待全部后台数据返回（包括出错）后完成初始化页面，方可移除蒙板（遮罩）。

反例：

同上，用户在数据请求没有完成时执行业务操作，导致部分数据与实际数据存在差异，却执行了后续的业务操作。

(十) 第三方对接规约

1、【推荐】对接过程中，如果需要对第三方提供的一些字段拼接内容作为自己的判断依据，推荐在字段的前后拼接固定长度的内容，再使用时截取指定长度字符串处理。

反例：

思普 PLM 对接中，使用了 PLM 的 clientId，并在其后面拼接了 - 登录来源，在使用时通过 split 来截取来源信息，后续使用过程中，发现 PLM 提供的 clientId 出现了自带一个 - 横杠的情况，导致 split 取出的信息错误，用户无法登陆。

(十一) git 日志提交规范

1、【强制】提交代码时应该对本次修改的主要内容进行简要说明
提交记录是方便查找对应修改内容的标记，是对接任者的负责。

反例：



优化b	cdh	2025/6/23 9:44
优化b	cdh	2025/6/23 9:41
优化b	cdh	2025/6/20 15:57
优化b	cdh	2025/6/20 11:05
优化b	cdh	2025/6/19 14:11
优化b	cdh	2025/6/19 13:36
优化	cdh	2025/6/18 18:40
优化	cdh	2025/6/18 18:08
优化	cdh	2025/6/17 12:41

2、【推荐】每日对开发完成的代码进行提交

多人协同时，尽早提交完成的代码，避免造成代码冲突；一个人开发时也可以尽早的备份修改内容，调整时方便追溯还原。

说明：

- 1、原则上要求提交每日的代码，提交后其他人同步下来后运行不报错
- 2、如果代码改动差异较大，提交后会对他人业务造成影响或无法启动的情况，则可以等改完再提交

四、业务规约

(一) 表字段

1、【强制】表必备字段：以下字段为表的基础字段，所有表需包含 红色为必须的字段，其他视业务需要添加

字段名	类型	长度	说明
id	varchar	32	主键、一般建议使用 32 位 UUID，不建议使用自
create_id	varchar	32	创建人 ID
create_name	varchar	64	创建人姓名
create_time	datetime		创建时间
create_full_path_id	text		创建人组织 id 全路径
create_full_path_name	text		创建人组织名称全路径
create_job_no	varchar	64	创建人工号
update_id	varchar	32	修改人 ID
update_name	varchar	64	修改人姓名
update_time	datetime		修改时间
update_full_path_id	text		修改人组织 id 全路径
update_full_path_name	text		修改人组织名称全路径
update_job_no	varchar	64	修改人工号
version	int	11	数据版本号
status	int	1	有效性，1 代表 true，有效；0 代表 false,无效。其他状态使用业务名_state 来表示对应的业务
deleted	int	11	物理删除状态，1 代表 true ,已删除，0 代表 false,未删除

(二) 开发设计

- 1、**【强制】**列表只查询列表要展示的数据，详情通过详情接口单独去调，不在列表中带出。

说明：单接口尽量只返回必要的数据，否则会造成查询过慢的问题。

- 2、**【强制】**记录被引用规范，由删除的业务 A 控制，删除时判断所有业务是否有用到业务 A 要删除的数据(接口由其他业务提供，业务 A 删除逻辑里逐一判断)。

- 3、**【强制】**所有导入模板每列值去前后空格、导入报错判断具体报错，所有导出模板每列宽度适宜、不需要每列去拉。

说明：导出模板宽度控制需要项目经理提供真实数据的建议列宽。

- 4、**【强制】**另外使用事务的地方需要考虑各方面的回滚方案，包括缓存回滚、搜索引擎回滚、消息补偿、统计修正等。

- 5、**【推荐】**业务复杂，要求实时的数据，维护一个统计表，所有涉及到的业务影响，都要去维护统计表。业务复杂，实时性不高的，可以定时器每天/每周/每月跑一次。表里面无用数据，定时器定期清理，比如消息提醒，只保留一周或者一个月。

说明：

- 1、具体的实时性要求由项目经理确认。

- 2、统计表应该在业务发生变更时进行更新，避免经常批量更新导致占用线程影响业务操作，可定期在夜间（选无业务发生时段）做一个统一更新，确保数据准确性。

- 6、**【强制】**所有附件下载，下载到本地必须使用文件上传时的名称。（2022-01-04 新增）

- 7、**【强制】**所有查询接口默认应做数据权限控制，非当前人数据权限范围内的数据不允许查询，如项目经理提出需要按特殊需求处理数据权限的，以项目经理需求为准。

- 8、**【强制】**所有系统定时任务，应该在前端页面中支持配置启动规则与停启。

说明：

- 1、硬编码定时任务规则到程序中会导致更新定时任务需要重启服务，非常不方便管理员进行维护。

- 2、使用 xxl-job 进行管理的话，需要注意选用 2.5 版本以上（选用支持 JDK8 的最新版），并注意作者更新，不要使用包含公开漏洞的版本，且 xxl-job 的端口不允许开放到公网，仅通过本机调用，调用账号密码应设置强密码。

- 9、**【强制】**所有列表页的字段，都应支持排序切换。（2024-01-20 新增）

说明：如涉及到跨库查询或计算项无法支持排序时，可与项目经理确认，去除该字段的排序功能

- 10、**【强制】**所有系统应接入操作日期记录，针对主要接口记录用户操作日志。（2025-

说明:

- 1、操作日期的库应该与业务库分离
- 2、具体哪些接口需要记录操作日期, 由项目经理指定
- 3、操作日志按国家规定至少保存 6 个月

10、**【强制】** 当系统存在的第三程序对接时, 应记录调用接口的详细信息。(2025-07-03 新增)

说明:

- 1、第三方系统的调用日志库应于业务库分离, 可与操作日期在同一个库
- 2、记录表结构, 记录调用的具体业务、调用接口地址、调用参数、返回参数、操作时间、第三方系统名称

```
SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;

-- -----
-- Table structure for sys_third_party_log
-- -----
DROP TABLE IF EXISTS `sys_third_party_log`;
CREATE TABLE `sys_third_party_log` (
  `id` int(11) NOT NULL AUTO_INCREMENT COMMENT '主键',
  `business_name` varchar(255) CHARACTER SET utf8mb4
  COLLATE utf8mb4_general_ci NOT NULL COMMENT '业务名称',
  `url` varchar(255) CHARACTER SET utf8mb4 COLLATE
  utf8mb4_general_ci NOT NULL COMMENT '调用的接口',
  `oper_param` varchar(7000) CHARACTER SET utf8mb4
  COLLATE utf8mb4_general_ci NOT NULL COMMENT '调用参数',
  `json_result` varchar(7000) CHARACTER SET utf8mb4
  COLLATE utf8mb4_general_ci NOT NULL COMMENT '返回参数',
  `oper_time` varchar(20) CHARACTER SET utf8mb4 COLLATE
  utf8mb4_general_ci NULL DEFAULT NULL COMMENT '操作时间',
  `third_system_name` varchar(255) CHARACTER SET utf8mb4
  COLLATE utf8mb4_general_ci NOT NULL COMMENT '第三方系统名
  称',
  PRIMARY KEY (`id`) USING BTREE
) ENGINE = InnoDB AUTO_INCREMENT = 7 CHARACTER SET =
  utf8mb4 COLLATE = utf8mb4_general_ci ROW_FORMAT =
  DYNAMIC;

SET FOREIGN_KEY_CHECKS = 1;
```

11、【推荐】系统包含 workflow 时，业务与 workflow 引擎应该解耦

目前开发的很多项目业务与 workflow 强耦合，当客户要求修改流程时，会导致很多代码需要跟着修改，及其不方便维护。

说明：

- 1、在项目评审时遇到业务与流程引擎强耦合时，需要建议项目进行修改方案。
- 2、若项目经理坚持强耦合的方式，需要同部门负责人一起讨论方案确认是否能解耦或改成其他的方案

五、MySQL 数据库

(一) 建表规约

- 1、【强制】表名、字段名必须使用小写字母或数字，禁止出现数字开头，禁止两个下划线中间只出现数字。数据库字段名的修改代价很大，因为无法进行预发布，所以字段名称需要慎重考虑。

说明：MySQL 在 Windows 下不区分大小写，但在 Linux 下默认是区分大小写。因此，数据库名、表名、字段名，都不允许出现任何大写字母，避免节外生枝。

正例：aliyun_admin, rdc_config, level3_name

反例：AliyunAdmin, rdcConfig, level_3_name

- 2、【强制】表名不使用复数名词。

说明：表名应该仅仅表示表里面的实体内容，不应该表示实体数量，对应于 DO 类名也是单数形式，符合表达习惯。

- 3、【强制】禁用保留字，如 desc、range、match、delayed 等，请参考 MySQL 官方保留字。

- 4、【强制】主键索引名为 pk_字段名；唯一索引名为 uk_字段名；普通索引名则为 idx_字段名。

说明：pk_ 即 primary key；uk_ 即 unique key；idx_ 即 index 的简称。

- 5、【强制】小数类型为 decimal，禁止使用 float 和 double。

说明：在存储的时候，float 和 double 都存在精度损失的问题，很可能在比较值的时候，得到不正确的结果。如果存储的数据范围超过 decimal 的范围，建议将数据拆成整数和小数并分开存储。

- 6、【强制】如果存储的字符串长度几乎相等，使用 char 定长字符串类型。

7、【强制】varchar 是可变长字符串，不预先分配存储空间，长度不要超过 5000，如果存储长度大于此值，定义字段类型为 text，独立出来一张表，用主键来对应，避免影响其它字段索引效率。

8、【推荐】表的命名最好是遵循“业务名称_表的作用”。

9、【推荐】库名与应用名称尽量一致。

10、【推荐】字段允许适当冗余，以提高查询性能，但必须考虑数据一致。冗余字段应遵循：

- 1) 不是频繁修改的字段。
- 2) 不是唯一索引的字段。
- 3) 不是 varchar 超长字段，更不能是 text 字段。

11、【推荐】单表行数超过 500 万行或者单表容量超过 2GB，才推荐进行分库分表。

说明：如果预计三年后的数据量根本达不到这个级别，请不要在创建表时就分库分表。

12、【参考】合适的字符存储长度，不但节约数据库表空间、节约索引存储，更重要的是提升检索速度。

13、【推荐】数据库 id 全部用无符号的自增，查询创建时间排序的直接用 id 排。

如果业务联表查询多，还是使用 uuid 形式保存主键

栏位	索引	外键	触发器	选项	注释	SQL 预览		
名				类型	长度	小数点	不是 null	
id				int	10	0	☒	 1

默认:

注释:

☒ 自动递增

☒ 无符号

☐ 填充零

栏位	索引	外键	触发器	选项	注释	SQL 预览
引擎:				InnoDB		
字符集:				utf8mb4 -- UTF-8 Unicode		
排序规则:				utf8mb4_general_ci		
自动递增:				1	修改自增起始值	
行格式:				DYNAMIC		
平均行长度:						
最大行:						
最小行:						
键块大小:						
数据目录:						
索引目录:						
分割区						

13、【推荐】SQL 性能优化的目标：至少要达到 range 级别，要求是 ref 级别，如果可以

是 **consts** 最好。

说明：1) **consts** 单表中最多只有一个匹配行（主键或者唯一索引），在优化阶段即可读取到数据。2) **ref** 指的是使用普通的索引（**normal index**）。3) **range** 对索引进行范围检索。

14、【强制】数据库修改需要记录脚本，后续更新都通过脚本更新原有数据库

举例：

```
ALTER TABLE `lims_task_allot` add `cancel_reason`  
varchar(255) DEFAULT NULL COMMENT '取消原因';
```

说明：通过脚本修改方便更新历史数据库

15、【强制】数据库在处理日期格式数据时不要使用不符合日期格式规范的数据

反例：

0000-00-00

1900-01-00

说明：错误的数据格式会导致数据库备份还原时报错，导致数据或者整表无法进行导入

(二) SQL 语句

1、【强制】不要使用 **count(列名)**或 **count(常量)**来替代 **count(*)**，**count(*)**是 SQL92 定义的标准统计行数的语法，跟数据库无关，跟 **NULL** 和非 **NULL** 无关。

说明：**count(*)**会统计值为 **NULL** 的行，而 **count(列名)**不会统计此列为 **NULL**值的行。

2、【强制】**count(distinct col)** 计算该列除 **NULL** 之外的不重复行数，注意 **count(distinct col1,col2)** 如果其中一列全为 **NULL**，那么即使另一列有不同的值，也返回为 0。

3、【强制】当某一列的值全是 **NULL** 时，**count(col)**的返回结果为 0，但 **sum(col)**的返回结果为 **NULL**，因此使用 **sum()**时需注意 **NPE** 问题。

正例：

可以使用如下方式来避免 **sum** 的 **NPE** 问题：**SELECT IFNULL(SUM(column), 0) FROM table;**

4、【强制】使用 **ISNULL()**来判断是否为 **NULL** 值。

说明：**NULL** 与任何值的直接比较都为 **NULL**。

- 1) **NULL<>NULL** 的返回结果是 **NULL**，而不是 **false**。
- 2) **NULL=NULL** 的返回结果是 **NULL**，而不是 **true**。
- 3) **NULL<>1** 的返回结果是 **NULL**，而不是 **true**。

反例：在 SQL 语句中，如果在 null 前换行，影响可读性。select * from table where column1 is null and column3 is not null; 而`ISNULL(column)`是一个整体，简洁易懂。从性能数据上分析，`ISNULL(column)` 执行效率更快一些。

5、**【强制】** 代码中写分页查询逻辑时，若 count 为 0 应直接返回，避免执行后面的分页语句。

6、**【强制】** 不得使用外键与级联，一切外键概念必须在应用层解决。

说明：（概念解释）学生表中的 student_id 是主键，那么成绩表中的 student_id 则为外键。如果更新学生表中的 student_id，同时触发成绩表中的 student_id 更新，即为级联更新。外键与级联更新适用于单机低并发，不适合分布式、高并发集群；级联更新是强阻塞，存在数据库更新风暴的风险；外键影响数据库的插入速度。

7、**【强制】** 禁止使用存储过程，存储过程难以调试和扩展，更没有移植性。

8、**【强制】** 数据订正（特别是删除或修改记录操作）时，要先 select，避免出现误删除，确认无误才能执行更新语句。

9、**【推荐】** in 操作能避免则避免，若实在避免不了，需要仔细评估 in 后边的集合元素数量，控制在 1000 个之内。

10、**【参考】** 因国际化需要，所有的字符存储与表示，均采用 utf8 字符集，那么字符计数方法需

说明： SELECT LENGTH("轻松工作"); 返回为 12

SELECT CHARACTER_LENGTH("轻松工作"); 返回为 4

如果需要存储表情，那么选择 utf8mb4 来进行存储，注意它与 utf8 编码的区别。

11、**【参考】** TRUNCATE TABLE 比 DELETE 速度快，且使用的系统和事务日志资源少，但 TRUNCATE 无事务且不触发 trigger，有可能造成事故，故不建议在开发代码中使用此语句。

说明：TRUNCATE TABLE 在功能上与不带 WHERE 子句的 DELETE 语句相同。

(三) ORM 映射

1、**【强制】** 在表查询中，一律不要使用 * 作为查询的字段列表，需要哪些字段必须明确写明。

说明：1) 增加查询分析器解析成本。

2) 增减字段容易与 resultMap 配置不一致。

3) 无用字段增加网络消耗，尤其是 text 类型的字段。

2、**【强制】** sql.xml 配置参数使用：#{}, #param# 不要使用\${} 此种方式容易出现 SQL 注入。

说明：

1、如果有必须使用\${}的场景，应该确保数据是由后端校验后填入（预防 SQL 输入），而非直接使用前端传入的数据

3、**【推荐】** 不要写一个大而全的数据更新接口。传入为 POJO 类，不管是不是自己的目标更新字段，都进行 update table set c1=value1,c2=value2,c3=value3; 这是不对的。执行 SQL 时，不要更新无改动的字段，一是易出错；二是效率低；三是增加 binlog 存储。

六、系统部署

(一) JVM 调优

2、**【强制】** 当用户提供的服务器性能，大于 4 核 8G 内存时，应该通过修改 Java 启动参数，实现 JVM 高性能调度。

关键参数说明

set port=8081 ：端口使用服务端口，可以自动关闭该端口进程

"srm-u9-api" ：设置批处理窗口的标题

java -server -Xms10g -Xmx10g ：设置 JVM 使用的最小及最大内存为 10G（根据实际服务器配置调整使用内存大小）

-XX:MetaspaceSize=256m -XX:MaxMetaspaceSize=384m ：每次内存使用扩展的大小。

-XX:ParallelGCThreads=4 ：CPU 线程占用数，只部署一个服务的话建议填服务器最大核心数，多个服务器试其他服务性能需求调整。

-XX:ConcGCThreads=1 ：回收线程占用数，一般试上面配置除以 4。

--jasypt.encryptor.password=8ac508125a57a3f3: jasypt 的加密盐值。

--logging.config=D:/Server/LxSRM_u9/logback.xml: 指定外部的日志配置文件，建议关闭控制台日志输出，需要的信息保存到日志文件中，可以显著提高系统运行效率。

几种配置的推荐参数

4 核 16g 推荐使用的配置

```
@echo off
%这里需要修改端口号%
set port=8081
for /f "tokens=1-5" %%i in ('netstat -ano^|findstr
":%port%") do (
echo kill the process %%m who use the port
taskkill /pid %%m -t -f
goto start
)
:start
%这里需要修改窗口标题，及执行的指令%
START "srm-u9-api" java -server -Xms10g -Xmx10g -
XX:MetaspaceSize=256m -XX:MaxMetaspaceSize=384m -
XX:+UseG1GC -XX:MaxGCPauseMillis=150 -
XX:InitiatingHeapOccupancyPercent=40 -
XX:G1ReservePercent=10 -XX:ParallelGCThreads=4 -
XX:ConcGCThreads=1 -XX:+DisableExplicitGC -
XX:+UseStringDeduplication -
XX:StringDeduplicationAgeThreshold=3 -jar srm-1.0-
SNAPSHOT.jar --
logging.config=D:/Server/LxSRM_u9/logback.xml
```

40 核 64G 内存推荐配置

```
@echo off
%这里需要修改端口号%
set port=8081
for /f "tokens=1-5" %%i in ('netstat -ano^|findstr
":%port%") do (
echo kill the process%%m who use the port
taskkill/pid%%m-t-f
goto start
)
:start
%这里需要修改窗口标题，及执行的指令%
START"srm-u8-api"java-server-Xms32g-Xmx32g-
XX:MetaspaceSize=256m-XX:MaxMetaspaceSize=512m-XX:+UseG1GC-
XX:MaxGCPauseMillis=200-
XX:InitiatingHeapOccupancyPercent=35-
XX:G1ReservePercent=15-XX:ParallelGCThreads=20-
XX:ConcGCThreads=5-XX:+DisableExplicitGC-
XX:+UseStringDeduplication-
XX:StringDeduplicationAgeThreshold=3-jar lx-srm-u8.jar--
jasypt.encryptor.password=8ac508125a57a3f35979f28e8367f8a8-
-logging.config=D:/Server/LxSRM/logback.xml
```

